

Matroid

拟阵

引言

拟阵 (Matroid) 是哈斯勒·惠特尼 (Hassler Whitney) 于 1935 年提出的一种抽象代数结构，旨在统一和推广关于独立性的概念，例如线性代数中的线性无关性和图论中的无环性。

拟阵为处理与独立性相关的优化问题提供了强大的理论工具，广泛应用于组合数学、图论、算法设计等领域，尤其在为贪心算法等优化方法提供数学理论支持方面发挥了重要作用。

定义

拟阵

一个拟阵 (Matroid) 可以表示为 $(E, \mathcal{I})$ ，其中：

- E 是一个有限集，称为 **基础集 (Ground Set)**。
- $\mathcal{I}$ 是 E 的子集族，称为 **独立集族 (Family of Independent Sets)**，其中的集合称为 **独立集 (Independent Set)**。有以下三个性质：
 - 非空性**：空集是独立的，即 $\emptyset \in \mathcal{I}$ 。
 - 遗传性**：独立集的任意子集也是独立集。若 $I \in \mathcal{I}$ ，则对于任意 $J \subseteq I$ ，都有 $J \in \mathcal{I}$ 。
 - 扩张性**：若 $I, J \in \mathcal{I}$ 且 $|I| < |J|$ ，则存在 $x \in J \setminus I$ ，使得 $I \cup \{x\} \in \mathcal{I}$ 。

如果一个形如 $(E, \mathcal{I})$ 的结构满足上述三个性质，则称其为一个拟阵。

基

基 (Basis) 是拟阵中极大的独立集，即无法再添加元素而保持独立性的独立集。所有基的集合称为 **基集族**，记为 $\mathcal{B}$ 。

性质：

- 等基数性**：所有基的大小都相同，称为拟阵的 **秩 (Rank)**。
- 扩张性**：任何独立集通过添加基中的元素都可以扩张为一个基。

圈

圈 (Circuit) 是拟阵中最小的依赖集，即其所有真子集都是独立的，但自身不是独立集，任意两个圈之间不存在包含关系。

秩

秩函数 (Rank Function) r 将基础集 E 的子集映射到非负整数。对于任意 $S \subseteq E$ ，定义为 S 中最大独立集的大小，即 $r(S) = |\mathcal{I}(S)|$ 。

性质：

- 非负性：对于任意 S ，有 $r(S) \geq 0$ 。
- 单调性：若 $S \subseteq T$ ，则 $r(S) \leq r(T)$ 。
- 次模性：对于任意 S, T ，有 $r(S \cup T) + r(S \cap T) \leq r(S) + r(T)$ 。

典型示例

1. 均匀拟阵 (Uniform Matroid)

定义：给定基础集 E 和非负整数 k ，均匀拟阵 U_k 的独立集族是所有大小不超过 k 的子集，表示为：

。

- 基 (Bases)：所有大小为 k 的子集。
- 圈 (Circuits)：所有大小为 $k+1$ 的子集。
- 秩 (Rank)：若 $k < |E|$ ，即独立集中最多能有 k 个元素。

2. 图拟阵 (Graphical Matroid)

定义：给定一个无向图 G ，图拟阵 $M(G)$ 的基础集是边集 E ，其独立集族是所有不包含环的边集，即所有的森林。

- 基：图中的生成树（在连通图的情况下）。生成树是极大的独立集，无法再增加边而不形成环。
- 圈：图中的简单环，去掉环中的任意一条边，剩余部分都为独立集。
- 秩：若 G 有 n 个顶点和 c 个连通分支，其中 c 是图的连通分支数。对于一个连通的无向图，其秩等于顶点数减一，即 $r = n - 1$ 。

3. 线性拟阵 (Linear Matroid)

定义：线性拟阵基于向量空间。给定向量空间 V ，基础集 E 是 V 中的一组有限向量，其独立集族是 E 中所有线性无关的向量子集。

- 基：极大的线性无关向量集，其大小等于向量空间的维数。
- 圈：最小的线性相关向量集合，其任意真子集都是独立的，而自身是线性相关的。
- 秩：线性拟阵的秩 r ，即向量空间的维数。独立集的大小不能超过向量空间的维数。

4. 划分拟阵 (Partition Matroid)

定义：将基础集 E 划分为不相交的子集 $E_1, E_2, \dots, E_k$ ，并为每个子集 E_i 指定一个非负整数 b_i 。划分拟阵的独立集族由满足每个部分选取元素数量不超过 b_i 的子集组成，表示为：

。

- 基：满足 $|I \cap E_i| \leq b_i$ 的独立集是划分拟阵的基。每个基在每个子集中选取了恰好 b_i 个元素。
- 圈：划分拟阵的圈是最小的依赖集，即包含至少一个元素数量超过 b_i 的子集。
- 秩：划分拟阵的秩为 $r = b_1 + b_2 + \dots + b_k$ ，即最大独立集的大小等于每个子集中允许选取的最大元素数的总和。

5. 有色拟阵 (Colored Matroid)

定义：有色拟阵是划分拟阵的一种特殊形式，其中每个元素都赋予了颜色。给定基础集 E 和颜色集 C ，每个元素 $e \in E$ 都与某个颜色 $c \in C$ 相关联。有色拟阵的独立集不仅需要满足普通拟阵的独立性条件，还必须遵守颜色上指定的限制，例如同一种颜色的元素在独立集中最多选取一定数量。

- 基**：有色拟阵的基是符合颜色限制和独立性条件的极大独立集。
- 圈**：圈是最小的依赖集，包含至少一个违反独立性或颜色限制的元素集合。
- 秩**：有色拟阵的秩是满足颜色限制条件下的最大独立集大小。它既依赖于拟阵的结构，也依赖于颜色限制的具体规定。

构造和运算

对偶

给定拟阵，其 **对偶拟阵** 定义为：

。

性质：

- 基**：对偶拟阵的基是 的基在基础集 中的补集。换句话说，如果 是 的基，那么 就是 的基。
- 秩函数**：对偶拟阵的秩函数为，其中 是 的子集。这意味着对偶拟阵的秩可以通过基础集的大小、原拟阵的秩以及从基础集中移除 后的秩来计算。
- 自反性**：对偶拟阵的对偶仍是原拟阵，即 。

示例：

对于一个无向图，图拟阵的对偶是由图的割集组成的拟阵。图拟阵的基是图中的生成树，而其对偶的基是图的割集，即将图分成两个不连通部分的最小边集。

例如，考虑一个简单的三角形图，其边集为。图拟阵的基是两条边的集合（如），而对偶拟阵的基是单条边的集合（即割集，如），因为移除其中的一条边就会将图分割为两个连通分支。

删除和收缩

删除 (Deletion)：

对于，拟阵删除后得到新的拟阵，其独立集族定义为：

。

可以看出，删除操作就是从拟阵中移除某些元素，并保留剩余元素形成的独立集，其保持原独立集不变，只是移除了元素。

收缩 (Contraction)：

对于，拟阵收缩后得到拟阵，其独立集族定义为：

收缩操作可以理解为将集合中的元素缩约，并考虑剩下的元素与 的基一起形成的独立集。收缩的结果依赖于集合的基，缩约后的独立集实际上是对原拟阵中更高秩的子集进行约简后得到的独立集。

示例 - 图拟阵：

- 删除**：在图拟阵中，删除操作即从图中删除一些边。一个图删除某条边后，考虑的是剩余边所形成的独立集，即那些不包含环的边集。例如，如果从一个三角形图中删除一条边，剩下的两个边仍然是一个森林。
- 收缩**：收缩操作则是将某条边收缩为一个顶点。对于图拟阵，收缩一条边相当于将这条边的两个顶点合并成一个

顶点，并删除该边，合并顶点后，图中的其他边仍然可以形成独立集。例如，在一个三角形图中，收缩任意一条边将把两个顶点合并成一个，剩下的两条边将构成一个新的拟阵。

拟阵和贪心

问题描述：

拟阵的应用之一是解决贪心算法中的最优化问题。具体而言，给定一个拟阵，其中 $\mathcal{B}$ 是基础集， $\mathcal{I}$ 是独立集族。对于每个元素 e ，赋予一个正整数权值 w_e ，目标是找到权值最大的独立集，形式化为：

显然，权值最大独立集必须是极大独立集。如果一个独立集 I 不是极大独立集，则存在一个可以加入 I 的元素 e ，且由于 $e \notin I$ ，加入该元素后权值会增加，说明 I 不是权值最大的独立集。

步骤

贪心算法求解权值最大独立集的步骤如下：

1. **元素排序**：将基础集 $\mathcal{B}$ 按照权值从大到小排序，记为序列 $e_1, e_2, \dots, e_n$ 。
2. **初始化**：设独立集 $I = \emptyset$ 。
3. **构建独立集**：依次考虑排序后的元素 e_i ，如果 $I \cup \{e_i\} \in \mathcal{I}$ ，则更新 $I = I \cup \{e_i\}$ 。
4. **输出结果**：最终的集合 I 即为权值最大的独立集。

复杂度分析：

设 n 为基础集的大小， $\mathcal{I}$ 表示判断一个集合是否为独立集的复杂度。贪心算法的时间复杂度为：

其中， $\mathcal{O}(n \log n)$ 是排序的复杂度， $\mathcal{O}(n)$ 是逐一判断独立性的复杂度。

▼ 备注

- 在图拟阵中，可以使用[并查集](#)来高效检测是否形成环，从而使 $\mathcal{I}$ 接近常数时间。
- 在线性拟阵中，独立性检测通常涉及矩阵运算，其复杂度依赖于具体实现方式。

正确性证明：

设 $\mathcal{M}$ 是一个拟阵， I 是一个独立集，且 I 是某个权值最大独立集 I^* 的子集。定义集合 $\mathcal{J}$ ，即所有加入 I 后，仍然使 $I \cup \{e\}$ 保持独立性的元素所构成的集合。

设 e 为 $\mathcal{J}$ 中权值最大的元素，则 $I \cup \{e\}$ 也是某个权值最大独立集的子集，证明如下：

假设 $I \cup \{e\}$ 不是任何权值最大独立集的子集，则存在一个权值最大的独立集 J ，且 $e \notin J$ 。

由于 J 是权值最大独立集，根据拟阵的 **扩张性**，存在 $f \in I$ 使得 $f \in J$ 。

利用 **扩张性**，不断将 f 加入 J ，最终构造出一个新的独立集 J' ，使得 $e \in J'$ 。

设 J' 为构造出的独立集，此时有 $e \in J'$ 。由于 e 为 $\mathcal{J}$ 中权值最大的元素，有 $w_e \geq w_f$ 。

因此， J' 的权值不小于 J 的权值，此时：

- 若 $J' = J$ ，则 J 不是权值最大独立集，与假设矛盾。
- 若 $J' \neq J$ ，则 J' 为权值最大独立集，且 J 为其子集，与假设 J 不是任何权值最大独立集的子集矛盾。

综上，假设不成立，即 $I \cup \{e\}$ 必须是某个权值最大独立集的子集，因此通过不断使用贪心策略，最终可以找到权值最大的独立集。

立集。

示例

最小生成树：

给定一个连通无向图，每条边都具有权值。目标为找到一棵生成树，使其包含所有顶点且总权值最小。

拟阵的构建：

为了将最小生成树问题形式化为拟阵问题，可以构建图拟阵：

- **基础集：**，即图中的所有边。
- **独立集族：**为所有不包含环的边集（即所有森林）。

贪心算法：

在图拟阵的框架下，[Kruskal 算法](#)是一个典型的基于拟阵理论的贪心算法，可以用于构建最小生成树。虽然 [Prim 算法](#)也是一种有效的贪心算法，同样能够找到最小生成树，但它并不严格依赖于拟阵的贪心。因此，在拟阵理论的讨论中，Kruskal 算法是主要的贪心算法实例。

- **Kruskal 算法：**
 1. **边排序：**将所有边按权值从小到大排序。
 2. **逐步选择：**依次选择权值最小的边，若加入后不形成环，则将其加入生成树。
 3. **终止条件：**重复上述过程，直到生成树包含 $n-1$ 条边。
- **Prim 算法：**
 - **原理：**Prim 算法通过从一个起始顶点开始，逐步扩展生成树，每次选择连接树内与树外的最小权值边。
 - 虽然 Prim 算法也是贪心的，但其选择策略不同于其他基于拟阵扩张性质的贪心算法。因此，在拟阵理论的严格意义下，Prim 算法不被视为典型的拟阵贪心算法。

拟阵交

对于定义在同一基础集 E 上的两个拟阵 $\mathcal{P}_1$ 和 $\mathcal{P}_2$ ，若 $\mathcal{P}_1 \cap \mathcal{P}_2$ 满足拟阵独立集族的三条性质，则称 $\mathcal{P}_1$ 和 $\mathcal{P}_2$ 的交。

注意：并非任意两个拟阵的交都是一个拟阵，只有当其独立集族的交集满足拟阵独立集族定义中的三条性质时，其交才构成一个拟阵。

问题描述

1. **最大独立集：**在 $\mathcal{P}$ 中找到最大的独立集（即具有最大基数的独立集）。
2. **加权最大独立集：**给定权值函数 w ，在 $\mathcal{P}$ 中找到权值和最大的独立集。

算法

无权版本：

1. **初始化：**选择一个初始独立集，通常设定 $I = \emptyset$ 。
2. **迭代：**
 - **构建交换图：**根据当前独立集 I 构建交换图。
 - **路径选择：**在交换图中，寻找从源点 s 到汇点 t 的增广路径。
 - **增广：**沿路径 P 从 s 到 t 遍历每一个节点：
 - 如果节点属于左部顶点（即 L 中的元素），则将该元素从 I 中移除。
 - 如果节点属于右部顶点（即 R 中的元素），则将该元素加入 I 中。
 - **重复：**更新独立集 I 后，重复上述步骤，直到无法找到新的增广路径为止。

3. **结果**：最终得到的独立集 即为拟阵交 中的一个最大独立集。

加权版本：

为了找到权值和最大的独立集，算法需要在增广路径的选择上进行优化。

1. **权值设置**：对于每个元素，定义其在交换图中的权值：

- **左部顶点**（ 中的元素）： 。
- **右部顶点**（ 中的元素）： 。

2. **路径选择**：在交换图 中，寻找一条从源点 到汇点 的 **增广路径**，使得沿路径进行增广操作后，独立集 的总权值增加最大。

- **增广条件**：路径 上加入的元素的权值总和大于移除的元素的权值总和，即 加入的元素移除的元素

3. **增广操作**：沿路径 从 到 遍历每一个节点：

- 如果节点属于左部顶点（即 中的元素），则将该元素从 中移除。
- 如果节点属于右部顶点（即 中的元素），则将该元素加入 中。

4. **迭代**：重复步骤 1 至 3，不断构建交换图并寻找增广路径，逐步优化独立集 的总权值。

5. **终止条件**：当无法在交换图中找到满足增广条件的路径时，算法终止。

6. **结果**：最终得到的独立集 即为拟阵交 中的一个 **权值最大独立集**。

复杂度：

- **增广次数**：设两个拟阵的最大秩分别为 和 ，则最大增广次数为 。
- **每次增广的复杂度**：
 - 构建交换图的复杂度为 ，其中 。
 - 寻找增广路径的复杂度取决于路径搜索策略，通常为 ，例如使用广度优先搜索。
- **总时间复杂度**：总体的时间复杂度为 ，其中 。

例题

最小生成树：

给定一个无向图，每条边 都有一个权值 。寻找一棵生成树，使其包含所有顶点且总权值最小。

- 详细介绍：[最小生成树](#)。
- 题目模板：[洛谷 P3366【模板】最小生成树](#)。

► 解题思路

Colorful Graph：

给定一张带有多颜色的无向图，每条边有一个颜色属性。寻找一个最大的边集，使得：

1. 所选边不形成任何环。
2. 每种颜色的边数不超过 条（ 为给定的正整数）。

► 解题思路

约束的资源分配问题：

在一个资源分配问题中，有一组资源 和一组项目 。每个项目 需要分配一定数量的资源，且每种资源的总分配量不能超过其供应量。

目标：寻找一个资源分配方案，使其满足所有项目需求且不超过资源供应量。

参考资料与注释

1. [Wikipedia - Matroid](#)
 2. [百度百科 - 拟阵](#)
 3. [洛谷 - 拟阵与最优化问题](#)
 4. [洛谷 - 从拟阵基础到 Shannon 开关游戏](#)
-

本页面最近更新：2024/10/23 20:33:19, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [yyyu-star](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用